

Escalamiento de Datos en Machine Learning

By PhD. Josafá Pontes

Escalamiento de Datos en Machine Learning

- Normalización y Estandarización
- Preparación de datos para modelos
- Implementación desde cero

¿Por qué escalar los datos?

- Algoritmos sensibles a la escala
 - Regresión lineal
 - Regresión logística
 - Redes neuronales
- Evita sesgos por magnitud
 - Variables con diferentes magnitudes
→ afectan el aprendizaje

Efecto de escalar los datos (2)

Mejora

- Convergencia del entrenamiento del modelo
- Estabilidad numérica
- Interpretación

Problema sin escalamiento

- Edad: 0–100
 - Ingresos: 0–30000
- 👉 El modelo “prioriza” ingresos solo por magnitud

Tipos de escalamiento

- Normalización (Min-Max)
- Estandarización (Z-score)

¿Qué es la normalización?

- Reescala datos al rango $[0, 1]$
- No asume una distribución

Fórmula Normalización

- $x' = (x - \min) / (\max - \min)$

Ejemplo Normalización

- Dataset:

$[[50, 30],$
 $[20, 90]]$

- Min-Max:

$[20, 50] \leftarrow [\text{min}, \text{max}]$ de 1ra columna

$[30, 90] \leftarrow [\text{min}, \text{max}]$ de 2da columna

- Normalizado:

$[[1, 0],$
 $[0, 1]]$

¿Qué es la estandarización?

- Centra datos en media = 0
- Escala a desviación estándar = 1
- Asume distribución normal

Fórmula Estandarización

- $x' = (x - \mu) / \sigma$
- $\mu = \text{suma } x_i / n$
- $\sigma = \text{raiz_cuadrada}(\text{sum}(x_i - \mu)^2 / (n - 1))$
- $\text{standardized_value_i} = (x_i - \mu) / \sigma$

Ejemplo Estandarización

- Dataset:

$[[50, 30], [20, 90], [30, 50]]$

- Media:

$[33.33, 56.67] \leftarrow$ por columna

- Std:

$[15.28, 30.55]$

- Resultado:

$[[1.09, -0.87],$

$[-0.87, 1.09],$

$[-0.22, -0.22]]$

Ventajas de la Estandarización

- Ideal para modelos lineales
- Funciona bien con distribuciones normales
- Reduce sesgo por escala

Uso de cada método

Método	Supuesto	Rango	Uso recomendado
Normalización	Ninguno	$[0,1]$	Datos no normales
Estandarización	Normal	$(-\infty, \infty)$	Datos gaussianos

Buenas prácticas

- Guardar:
 - Min y Max
 - Media y desviación
- Aplicar mismos valores a nuevos datos

Ejercicio 1

- Explica:
 - ¿Por qué el escalamiento afecta a redes neuronales?
 - ¿Qué ocurre si no escalas los datos?

Ejercicio 2

- Calcular normalización manualmente

DataSet = $\begin{bmatrix} 10 & 100 \\ 20 & 200 \end{bmatrix}$

Obtener:

- Min-Max
- Normalización

¿Por qué el escalamiento afecta a redes neuronales? (1)

- Las redes neuronales aprenden mediante gradientes (descenso por gradiente). El problema es que esos gradientes dependen directamente de la escala de los datos de entrada.

¿Por qué el escalamiento afecta a redes neuronales? (2)

- ◆ 1. Influencia desigual de las variables
- Si una variable está en rango:
 - Edad → 0–100
 - Ingresos → 0–30000
- 👉 La red asigna más peso a Ingresos, no porque sea más importante, sino porque tiene mayor magnitud.

¿Por qué el escalamiento afecta a redes neuronales? (3)

◆ 2. Problemas en el entrenamiento (gradientes)

- Los pesos se actualizan proporcionalmente a los valores de entrada
- Valores grandes $\rightarrow$ gradientes grandes
- Valores pequeños $\rightarrow$ gradientes pequeños
- 👉 Resultado:
- Entrenamiento inestable
- Oscilaciones o divergencia

¿Por qué el escalamiento afecta a redes neuronales? (4)

◆ 3. Activaciones saturadas (muy importante)

Funciones como Sigmoid y Tanh se saturan con valores grandes:

$$\text{Sigmoid}(x \gg 0) \approx 1$$

$$\text{Sigmoid}(x \ll 0) \approx 0$$

👉 En saturación:

$$\text{Gradiente} \approx 0$$

❌ La red deja de aprender (vanishing gradient)

¿Por qué el escalamiento afecta a redes neuronales? (5)

◆ 4. Convergencia más lenta

Sin escalamiento:

El espacio de optimización es “alargado”

El descenso por gradiente zigzaguea

👉 Con escalamiento:

Superficie más “esférica”

Convergencia más rápida



Resumen

Las redes neuronales son muy sensibles a la escala

Escalar datos:

Mejora la estabilidad numérica del entrenamiento

Acelera el entrenamiento

Evita saturación

! En práctica: siempre escalar antes de entrenar

¿Qué ocurre si NO escalas los datos? (1)

1. Entrenamiento lento

- Más épocas necesarias
- Mayor costo computacional

2. Modelo subóptimo

- Aprende relaciones incorrectas
- Pesos mal ajustados



¿Qué ocurre si NO escalas los datos? (2)



3. Inestabilidad numérica

- Gradientes muy grandes → overflow
- Gradientes muy pequeños → underflow



4. Dominancia de ciertas variables

- Variables grandes “dominan” el aprendizaje
- Se pierde información relevante de otras variables



¿Qué ocurre si NO escalas los datos? (3)



5. Dificultad en inicialización

- Pesos iniciales dejan de ser adecuados
- Más sensibilidad a hiperparámetros



Resumen

Las redes neuronales son muy sensibles a la escala

Escalar datos:

Mejora la estabilidad

Acelera el entrenamiento

Evita saturación

! En práctica: siempre escalar antes de entrenar

Ejercicio 2 - Calcular normalización (1)

DataSet = [[10, 100], [20, 200]]

MinMax=[[10, 20],[100, 200]]

$x' = (x - \min) / (\max - \min)$ # Normalización

Fila 1: [10, 100]

Columna 1:

$[10 - 10] / [20 - 10] = 0/10 = 0$

Columna 2:

$[100 - 100] / [200 - 100] = 0/100 = 0$

Resultado: [0, 0]

Ejercicio 2 - Calcular normalización (2)

DataSet = [[10, 100], [20, 200]]

MinMax=[[10, 20],[100, 200]]

$x' = (x - \min) / (\max - \min)$ # Normalización

Fila 2: [20, 200]

Columna 1:

$$[20-10] / [20-10] = 10/10 = 1$$

Columna 2:

$$[200-100] / [200-100] = 100/100 = 1$$

Resultado: [1, 1]

Ejercicio 2 - Calcular normalización (3)

Resultado Final Normalizado:

$[[0, 0], [1, 1]]$

Conclusión

El valor mínimo $\rightarrow 0$

El valor máximo $\rightarrow 1$

Todo queda reescalado en $[0,1]$